2026-10-03 Fig1 배치 f2_ — 흉내 0단계(P0) 결과 + 10-02 정지 사고 원인

스펙 docs/superpowers/specs/2026-10-02-fig1-latent-design.md §4 · 코드 feat/fig1-latent 9183a0f (학습 코드는 2986481 과 같음) · Windows LICS-SERVER 2026-10-03 14:27–15:53 · 원자료 runs/2026-10-02_fig1/out/ · 이 문서 = runs/2026-10-02_fig1/report/make_report_p0.py 가 로그에서 직접 읽어 만듦

0. 요약

- **P0 판정: 불통과 (0/5 시드, 필요 3)** — 학생 평균 도착 45.2 % · 배끼리 충돌 51.5 % (기준 시드마다 ≥ 80 % · ≤ 15 %). 스펙 §4 대로 재시도 없이 멈춤, **phase1(trunk·코덱·갈래) 안 돌림**
- 학생은 안 피하는 goal 배(도착 50.3 · 충돌 49.7 %) 수준. 선생님 vo56h150 90.7 · 8.6 %, 10-02 i_ 학생(타각 행동) 71.1 · 28.7 % 보다 낮음
- DAgger 로그의 '선생님과 일치 98.1 %' 는 라벨 불균형 때문: 선생님 라벨의 약 89 % 가 '그대로 감(0)'. 피함 라벨에서 학생 일치는 68–73 %, 변침 크기는 선생님의 절반쯤 (§3)
- 주 평가는 학생의 **샘플 행동**으로 몲 — 침로 명령 잡음 표준편차 0.33(≈ 39°, DAgger 는 logstd 를 학습 안 함). 진단 창에서 같은 학생을 평균 행동(잡음 없음)으로 몰면 도착 41.4 → 65.8·68.5 %, 충돌 49.8 → 29.0·24.5 % (같은 창 선생님 84.1 · 8.6 %). 잡음을 빼도 P0 기준에는 못 미침 (§3, 진단 — 판정 아님)
- **10-02 f_ 정지·TDR 원인 = 실험이 아니라 메모리 설정 두 가지**(§4): ① config.py 기본 할당기 설정이 DAgger 캐시를 14.2 GB 까지 불림(실사용 3.2 GB) ② trunk 학습은 프로세스당 8.8 GB → GPU당 2개 불가. f2_ 는 환경변수 하나 + GPU당 개수만 바꿔 멈춤 없이 끝남

1. 무엇을 돌렸나

- f2_ = 10-02 f_ 의 재실행. 실험 설정·기준은 Python/runs_fig1/2026-10-02_fig1/_run_f.sh 그대로(목표 침로 행동 · ARPA@56·메시지 없음 · 선생님 vo56h150 · DAgger 3000결정 · 시드 43–47). 접두어만 f2_(HEAD 가 9183a0f 로 바뀌어 스크립트 규칙대로 새 접두어)
- 바꾼 것은 실행 환경뿐: PYTORCH_CUDA_ALLOC_CONF=garbage_collection_threshold:0.3(메모리 할당기 — 계산 결과와 무관) · phase0 GPU당 2개 / phase1 GPU당 1개 (report/diag/f2_launch.sh)
- 순서: smoke(preflight 14종 강제, 통과) 14:27–14:31 → DAgger 5시드 14:31–15:19 → 학생 주 평가 15:19–15:53 → P0 표
- P0 = 흉내만 낸 학생(PPO 전)을 주 평가(256 env · 10000결정 · burn-in 2400 · 평가 시드 999, 5칸 [run] 줄 확인)로 채점. 기준은 결과 전에 고정

2. P0 결과

시드	도착 %	배끼리 충돌 %	벽 %	시간초과 %	통과거리 m	COLREGs(옛) %	연료(도착 ep)	방향 바꾼 총량 °	DAgger 마지막 창 학생 도착 · 충돌 · 일치 %	P0
s43	44.4	52.3	0.0	3.3	23.3	32.1	1465	317	47.1 · 52.9 · 98.2	불통과
s44	46.7	50.1	0.0	3.3	23.2	31.4	1493	329	63.4 · 36.6 · 98.4	불통과
s45	44.3	52.5	0.0	3.3	23.5	31.7	1479	318	51.8 · 48.2 · 97.6	불통과
s46	44.2	52.7	0.0	3.2	23.2	31.7	1474	317	46.7 · 53.3 · 97.8	불통과
s47	46.3	50.1	0.0	3.5	23.2	31.8	1474	318	53.7 · 46.3 · 98.3	불통과
f2_ 학생 평균	45.2	51.5	0.0	3.3	23.3	31.7	1477	320		
i_ 학생 평균 (10-02)	71.1	28.7	0.0	0.2	22.1	38.2	987	115		
선생님 vo56h150	90.7	8.6	0.0	0.6	23.2	34.6	1000	92		
규칙 배 vo56	90.0	9.3	0.0	0.7	23.0	34.7	996	90		

규칙 배 goal(안 피함)	50.3	49.7	0.0	0.0	19.3	42.9	893	5		
학습 t_off	53.1	42.2	1.3	3.4	27.4	48.8	1449	467		

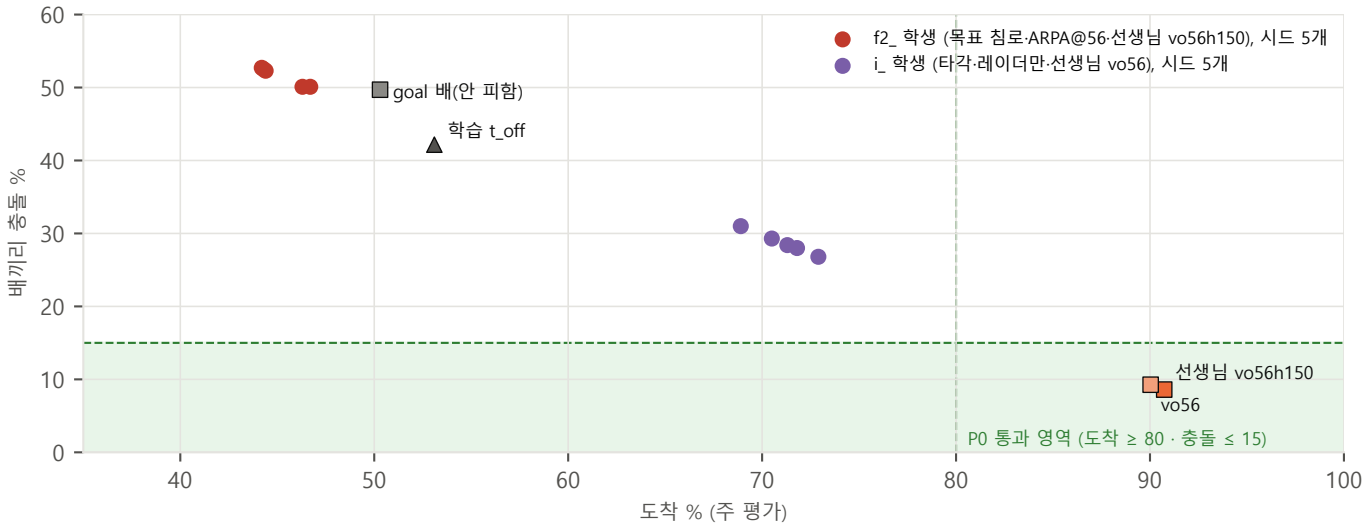


그림 1. 주 평가 도착 vs 배끼리 충돌. 빨강 = f2_ 학생(시드 5개), 보라 = 10-02 i_ 학생, 네모 = 규칙 배 평균(평가 시드 3개), 세모 = 학습 t_off 평균. 초록 = P0 통과 영역.

- 시드 간 차이 작음(도착 44.2–46.7 %) → 시드 운이 아니라 5개 모두 같은 양상
- 방향 바꾼 총량 320°(선생님 92°, goal 배 5°) · 연료 1477(선생님 1000) — 학생이 많이 흔들림
- i_ 학생과는 행동(타각 vs 목표 침로)·입력(레이더만 vs ARPA@56)·선생님(vo56 vs vo56h150)이 다 달라 어느 것 때문인지는 이 비교로 못 가림

3. 진단 — 왜 '일치 98 %' 인데 성적이 낮은가 (측정만, 판정 아님)

> 아래는 P0 뒤에 따로 잰 것(`report/diag/label_stats.py`, 학습·저장 없음). env 128개 · reset 뒤 300결정 버리고 잼 — 주 평가(256 env · 10000결정)와 창이 달라 숫자를 주 평가와 직접 비교하지 말 것. 시드 1–2개.

3-1. 선생님 라벨 분포와 조건부 일치 (300결정 창)

창 (300결정 × 2048척)	56 m 안에 배 있음 %	피함 라벨 (≠ 0) %	일치 전체 %	일치: 라벨 0 %	일치: 피함 라벨 %	방향 같음: 피함 라벨 %	|라벨| 평균 (피함)	|학생 평균 행동|(피함)	실행 잡음 표준편차
선생님이 몲 · 학생 s43	57.7	10.5	96.8	99.5	73.3	82.2	0.210	0.100	0.329
학생 s43 이 몲 (샘플)	51.5	10.0	96.2	99.3	67.8	78.5	0.233	0.115	0.329
학생 s44 이 몲 (샘플)	48.5	9.5	96.6	99.5	69.2	84.1	0.239	0.124	0.328

- 일치 = |tanh(학생 평균 a0) − 선생님 a0| < 0.2 (DAgger 로그와 같은 정의). a0 = 목표 방위 기준 침로 변경량 / 120°
- 라벨 대부분이 0 → 항상 0 을 내도 일치가 90 % 안팎. DAgger 로그의 일치·손실(−4.27 로 평평)은 학생이 피하는지 알려 주지 못함

3-2. 같은 학생을 샘플 행동 vs 평균 행동으로 몰기 (2500결정 창)

누가 몲 (2500결정 × 2048척)	끝난 ep	도착 %	배끼리 충돌 %	피함 라벨 %	일치: 피함 라벨 %
학생 s43 · 샘플 행동(주 평가와 같은 방식)	4370	41.4	49.8	6.0	66.1
학생 s43 · 평균 행동(잡음 없음)	5158	65.8	29.0	8.0	75.1
학생 s44 · 평균 행동(잡음 없음)	4980	68.5	24.5	7.5	79.0
선생님 vo56h150	4459	84.1	8.6	6.5	74.0

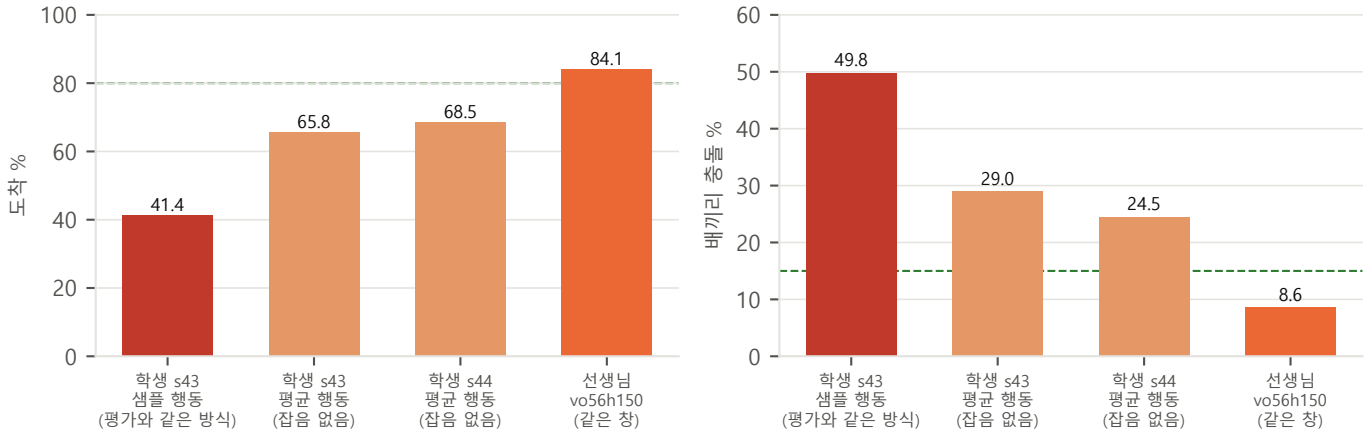


그림 2. 진단 창(2500결정)의 도착·충돌. 점선 = P0 기준(참고용 — 이 창은 P0 판정 창이 아님).

- 샘플 행동 → 평균 행동: 도착 +26 pp 안팎. 실행 잡음이 성적을 크게 깎음. `eval/eval_ckpt.py` 는 샘플 행동으로 몲(학습 배 평가와 같은 방식)
- 평균 행동으로도 선생님과 도착 18 pp 안팎 차이 → 잡음만의 문제가 아님(피함 라벨 일치 75–79 %)
- 확인 안 한 것: 잡음을 줄였을 때 주 평가 수치, 라벨 불균형을 보정했을 때 효과, i_ 대비 어느 변경(행동·입력·선생님)이 원인인지

4. 10-02 f_ 정지 사고 — 원인 측정과 조치

- 경과: 10-02 17:08 DAgger 5시드 시작 → 17:12–17:15 무진행(GPU 1·2·3 = 15.7–15.8 GB) → s47 만 22:44 완료(333분) → 22:45 nvlddmkm 153 ×5(TDR), s43·s44 `CUDA error: unknown error` → 10-03 00:04 서버 비정상 종료·00:07 재부팅 (Kernel-Power 41, 종료 원인 미확인). P0 판정 없이 끝남
- 재현: 10-03 단일 프로세스로도 같은 정지(200결정 뒤 무진행, GPU 15.9 GB) → 여러 개를 한 GPU 에 올린 탓만이 아님

프로세스	할당기 설정 (PYTORCH_CUDA_ALLOC_CONF)	실사용 최대 GB	reserved 최대 GB	진행
DAgger	기본값(config.py) `max_split_size_mb:128`	2.87 (추적 중 최대)	**14.25**	200결정 / 1.8분 — 이후 15분 넘게 무진행(수동 종료)
DAgger	`max_split_size_mb:128,garbage_collection_threshold:0.3`	2.89 (추적 중 최대)	**11.41**	200결정 / 2.0분 — 135 s 에 11.41 GB 로 자동 중단
DAgger	`garbage_collection_threshold:0.3` ← f2_ 에 씀	3.23	4.23	700결정 / 8.1분
DAgger	`max_split_size_mb:16384`	3.23	4.23	700결정 / 8.2분
trunk 학습	기본값	7.63	8.79	0.33M 결정, 2211 dec/s
trunk 학습	`garbage_collection_threshold:0.3` ← f2_ 에 씀	7.63	8.79	0.33M 결정, 2216 dec/s
trunk 학습	`max_split_size_mb:16384`	7.63	8.79	0.33M 결정, 2243 dec/s

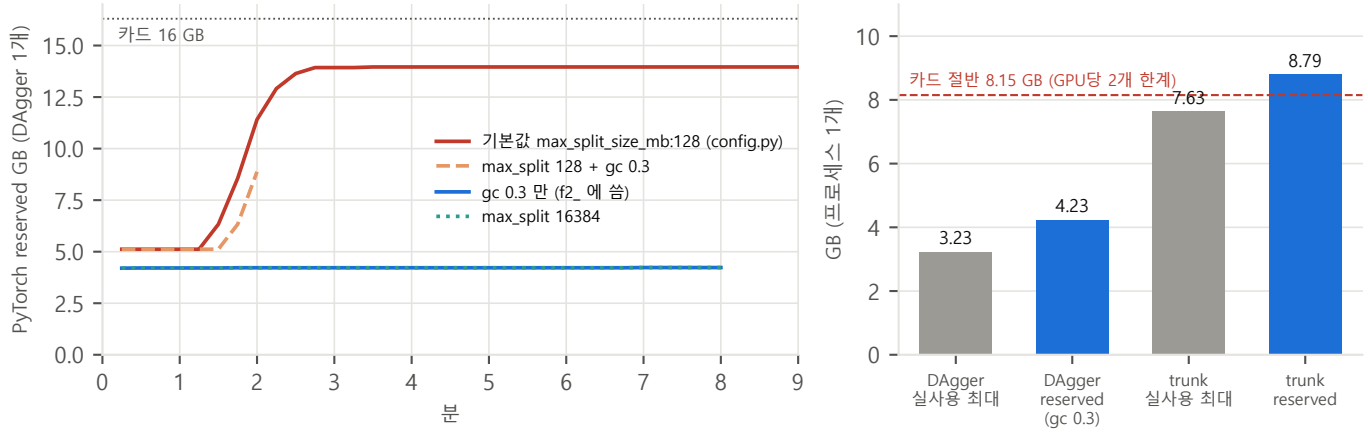


그림 3. 왼쪽 = DAgger 프로세스 1개의 PyTorch reserved 메모리(할당기 설정별). 오른쪽 = 프로세스당 실사용 최대와 reserved(gc 0.3). trunk 는 설정과 무관하게 8.8 GB.

- 원인 ①: `config.py` 의 기본 `max_split_size_mb:128` 아래에서 DAgger(ON·파트너 8·미니배치 2048)의 캐시가 실사용의 4배 넘게 쌓임. 이 설정만 빼면 4.2 GB 고정·정상 속도. 할당기 설정은 메모리 관리만 바꿈(계산 결과 불변) → 코드·HEAD 안 바꾸고 환경변수로 덮어씀
- 원인 ②: trunk 학습 실사용 최대 7.63 GB(할당기와 무관한 실제 수요). 스크립트 선언값 3000 MB · GPU당 2개면 16 GB 초과 → phase1 은 GPU당 1개로 띄우게 해 둠(이번엔 P0 불통과라 실행 안 됨)
- f2_ 실행 중 무진행·TDR 없음. DAgger 중 본 GPU 사용량 4.6–9.6 GB(14:57 표본 1회, 다른 계정 작업 포함)

5. 읽을 때 주의

> - P0 판정 근거는 §2 주 평가뿐. §3 은 다음 결정을 위한 진단(창·시드 수가 다름)
> - §3 의 '잡음' 설명은 측정된 차이(샘플 vs 평균 행동)까지만 근거 있음. PPO 를 붙이면 어떻게 되는지는 안 돌려 봄
> - CLAUDE.md 에 '서버 불안정으로 다른 기계 실행'이 적혀 있음 — 다른 기계에서 f_ 가 돌았다면 이 f2_ 와 같은 설정·시드의 중복 실행임(결과 대조 가능)
> - 10-02 f_ 의 s47 DAgger 체크포인트(정지 중 완료)는 이번에 안 씀 — f2_ 는 5시드 전부 새로 흉내

6. 다음 — 저자 결정

- 스펙 §4: P0 불통과면 재시도 없이 보고. 기준·설정 안 바꿈. phase1 은 대기(C:₩workWf2_launch.sh 는 P0 통과 기록이 있어야 phase1 로 감)
- 결정할 것(제안 아님, 갈림길만): 잡음(logstd)·평가 방식 / 라벨 불균형 / 행동 방식(목표 침로) 중 무엇을 바꿔 다시 흉내 낼지, 또는 흉내 없이 trunk 로 갈지